Memory Notes

Simplified DFI 3.1 Simplified edition - DFI Specification v3.1 (Denali Software)

October 4, 2026

Table of Contents

1 Simplified Dfi 3 1 Index....13

2 Introduction....14

2.1 What DFI is....14

2.2 Why the seam exists....15

2.3 Key features of DFI 3.1....16

2.4 Edition lineage....17

3 Organization....18

3.1 Sub-interface map for DFI 3.1....18

3.2 Clocking....19

3.3 Frequency ratio and the _pN / _wN replication rule....20

3.4 PHY data slices and byte lanes....21

3.5 Simplified teaching model....22

4 Acronyms....23

5 Control Interface....25

5.1 Command/address signals....26

5.2 DRAM enable and select signals....28

5.3 LPDDR3/LPDDR2 CA-bus mapping....29

5.4 Control timing parameters....30

6 Write Data Interface....31

6.1 Write data signals....32

6.2 dfi_wrdata_en behavior....34

6.3 Write data mask and DBI....35

6.4 Write chip select....36
6.5 Write data timing parameters....37
6.6 Write data programmable parameters....38
7 Read Data Interface....39
7.1 Read data signals....40
7.2 dfi_rddata_en and dfi_rddata_valid....42
7.3 Read chip select....43
7.4 LPDDR2 data-not-valid....44
7.5 Read data timing parameters....45
8 Update Interface....46
8.1 MC-initiated update....47
8.2 PHY-initiated update....48
8.3 Update timing parameters....49
8.4 Simultaneous requests....50
9 Status Interface....51
9.1 Initialization and configuration signals....52
9.2 Parity and alert signals....54
9.3 Status timing parameters....55
10 Training Interface....56
10.1 Read training signals....57
10.2 Write leveling signals....59
10.3 CA training signals (LPDDR3)....60
10.4 PHY-requested training in non-DFI training mode....61

10.5 Common training controls....62
10.6 Training timing parameters....63
10.7 Training programmable parameters....65
11 Low Power Control Interface....66
11.1 Low power signals....67
11.2 dfi_lp_wakeup encoding....68
11.3 Low power handshake....69
11.4 Low power timing parameters....70
12 Error Interface....71
12.1 Error signals....72
12.2 Error timing....73
12.3 Spec-defined error codes....74
13 Reset and Initialization....75
13.1 Default values before dfi_init_complete....76
13.2 The dfi_init_start / dfi_init_complete handshake....77
13.3 Frequency-change reuse of the init signals....78
13.4 Initialization timing parameters....79
14 Training in DFI 3.1....80
14.1 Which training is used where....81
14.2 DFI training mode....82
14.3 Read training sequence....83
14.4 Write leveling sequence....84
14.5 CA training sequence (LPDDR3)....85

14.6 Periodic vs full training....86
14.7 Non-DFI training mode and PHY-requested training....87
15 Command Mapping....88
15.1 DDR3 command encoding on DFI....89
15.2 LPDDR3/LPDDR2 CA-bus mapping....90
15.3 Chip select and enable usage....91
15.4 Command timing relationship....92
16 Write Carriage....93
16.1 Write timing parameters in use....94
16.2 Single write transaction....95
16.3 Contiguous vs non-contiguous writes....96
16.4 Write chip select timing....97
16.5 DBI and CRC notes....98
17 Read Carriage....99
17.1 Read timing parameters in use....100
17.2 Single read transaction....101
17.3 Contiguous vs non-contiguous reads....102
17.4 Per-slice independence....103
17.5 Read chip select timing....104
17.6 LPDDR2/LPDDR3 mode-register reads....105
18 Walkthroughs....106
18.1 Teaching model....107
18.2 Walkthrough 1: activate -> read -> data return....108

18.3 Walkthrough 2: activate -> write with preamble and mask....109
18.4 Walkthrough 3: back-to-back reads on different rows....110
19 Parameter Families....111
19.1 PHY-defined vs MC/system-defined parameters....112
19.2 Parameter families....113
19.3 Minimum and maximum conventions....114
19.4 Frequency-ratio timing....115
20 Ratio and Phasing....116
20.1 Matched frequency (1:1)....117
20.2 1:2 frequency ratio....118
20.3 1:4 frequency ratio....119
20.4 Read data rotation....120
20.5 Scheduling impact....121
20.6 Practical note for scoria....122
21 Parameter Tables....123
21.1 Programmable parameters....128
22 Low Power Behavior....129
22.1 Low-power request split....130
22.2 Typical sequence....131
22.3 Relationship to DRAM self-refresh....132
22.4 Scoria note....133
23 Update Behavior....134
23.1 MC-initiated update....135

23.2 PHY-initiated update....136
23.3 Idle-bus definition....137
23.4 Update after self-refresh exit....138
24 Frequency Change....139
24.1 Acknowledged frequency change....140
24.2 Not-acknowledged frequency change....141
24.3 Stable levels during the change....142
24.4 Scoria note....143
25 Refresh Interaction....144
25.1 Refresh and training....145
25.2 Refresh and update....146
25.3 Refresh and low power....147
25.4 Scoria note....148
26 Revision Deltas....149
26.1 What carried over unchanged from v2.1.1....150
26.2 v2.1.1 -> v3.0 changes....151
26.3 v3.0 -> v3.1 changes....152
26.4 Brief v4.0 preview....153
26.5 What scoria leaves out....154
27 DFI 3.1 in the Scoria Controller....155
27.1 Why scoria chose v3.1....156
27.2 What scoria inherits....157
27.3 What scoria implements from v3.1....158

27.4 What scoria does not implement.....159
27.5 Known documentation corrections.....160
27.6 Study corrections carried into this book.....161

List of Figures

No figures in this document.

List of Tables

Table 1: DFI 3.1 sub-interfaces....18
Table 2: Acronyms used in this book....23
Table 3: Suffix notation....24
Table 4: Control interface command and address signals....26
Table 5: Control interface enable and select signals....28
Table 6: Control interface timing parameters....30
Table 7: Write data interface signals....32
Table 8: Write data interface timing parameters....37
Table 9: Write data programmable parameters....38
Table 10: Read data interface signals....40
Table 11: Read data interface timing parameters....45
Table 12: MC-initiated update signals....47
Table 13: PHY-initiated update signals....48
Table 14: Update interface timing parameters....49
Table 15: Status interface initialization and configuration signals....52
Table 16: Status interface parity and alert signals....54
Table 17: Status interface timing parameters....55
Table 18: Training operations in DFI 3.1....56
Table 19: Read training signals....57
Table 20: Write leveling signals....59
Table 21: CA training signals....60
Table 22: PHY-requested training signals....61
Table 23: Common training control signals....62
Table 24: Training interface timing parameters....63
Table 25: Training programmable parameters....65
Table 26: Low power control interface signals....67
Table 27: Low power wakeup encoding....68
Table 28: Low power control interface timing parameters....70
Table 29: Error interface signals....72
Table 30: Error interface timing parameter....73
Table 31: DFI 3.1 error codes....74
Table 32: Initialization handshake states....77
Table 33: Initialization timing parameters....79
Table 34: Training operations by memory type....81
Table 35: DDR3 command encoding on DFI control signals....89

Table 36: LPDDR3/LPDDR2 CA mapping onto dfi_address....90
Table 37: Write carriage timing parameters....94
Table 38: Write data stream patterns....96
Table 39: Read carriage timing parameters....100
Table 40: Timing parameter ownership....112
Table 41: Timing parameter families....113
Table 42: Complete DFI 3.1 timing parameter table....123
Table 43: DFI 3.1 programmable parameters....128
Table 44: v3.0 additions over v2.1.1....151
Table 45: v3.1 additions over v3.0....152
Table 46: Scoria DFI 3.1 implementation highlights....158
Table 47: DFI 3.1 features deliberately unimplemented in scoria....159

List of Waveforms

No waveforms in this document.

1 Simplified Dfi 3 1 Index

Generated: 2026-10-20

2 Introduction

2.1 What DFI is

The DDR PHY Interface (DFI) is a digital boundary between a DDR memory controller (MC) and the physical plane (PHY) that drives the actual DRAM pins. The MC thinks in commands, addresses, banks and bursts; the PHY turns those abstract transactions into edge-aligned clocks, strobes and data on the DRAM bus. DFI exists so that controller architects and PHY architects can work against a common contract instead of custom-wiring every MC/PHY pair.

At its simplest, DFI carries:

- Control: command/address signals such as `dfi_cs_n`, `dfi_ras_n`, `dfi_cas_n`, `dfi_we_n`, `dfi_address`, `dfi_bank`, plus DRAM-specific enables like `dfi_cke` and `dfi_odt`.
- Write data: `dfi_wrdata`, `dfi_wrdata_en`, `dfi_wrdata_mask`.
- Read data: `dfi_rddata`, `dfi_rddata_valid`, plus `dfi_rddata_en` from the MC to tell the PHY how many words to expect.
- Status, update, training, low-power and error sidebands that let the PHY and MC coordinate initialization, calibration, power states and error reporting.

DFI 3.1 supports DDR4, DDR3, DDR2, DDR1, LPDDR3, LPDDR2 and LPDDR1 DRAMs. Not every signal is required for every DRAM class; the controller for a given memory type drives only the subset it needs.

2.2 Why the seam exists

The MC and PHY run at different abstraction levels and often at different clock frequencies:

- The MC schedules commands and tracks bank state in its own clock domain.
- The PHY retimes commands to the DRAM clock, generates DQS, samples read data, and absorbs the analog timing complexity of the memory channel.

DFI decouples these two domains. The MC can issue a write command and trust that, `tphy_wrlat` cycles later, the PHY is ready for the data. The PHY can assert `dfi_rddata_valid` when it has captured read data, and the MC samples it in its own domain. Timing parameters such as `tctrl_delay`, `tphy_wrlat`, `tphy_wrdata`, `trddata_en` and `tphy_rdlat` are the contract that lets both sides agree on when information is valid without sharing a single clock.

2.3 Key features of DFI 3.1

- Frequency ratios of 1:1, 1:2 and 1:4 between the MC clock (DFI clock) and the PHY clock (DFI PHY clock). Commands are replicated per phase with a `_pN` suffix; read data words use `_wN`.
- A formal initialization and frequency-change protocol over `dfi_init_start` and `dfi_init_complete`.
- Separate low-power request signals for the control and data paths (`dfi_lp_ctrl_req` and `dfi_lp_data_req`), new in 3.1.
- PHY-requested training in non-DFI training mode (`dfi_phylvl_req_cs_n`, `dfi_phylvl_ack_cs_n`), new in 3.1.
- LPDDR3 CA training (`dfi_calvl_*`), new in 3.1.
- Error reporting (`dfi_error`, `dfi_error_info`) and parity/CRC/alert signals, inherited from 3.0 and mostly relevant to DDR4 systems.

2.4 Edition lineage

This book focuses on the subset of DFI 3.1 that a DDR3/LPDDR3 controller uses. The scoria controller inherited much of its datapath and command scheduling from a v2.1.1 baseline, then adopted v3.1 for its leveling and initialization model. Chapter 7 records exactly what changed and what scoria deliberately does not implement.

Source: DFI Specification v3.1 sections 1.0, 2.0

3 Organization

3.1 Sub-interface map for DFI 3.1

DFI 3.1 organizes its signals into eight sub-interfaces. Each interface has a direction (MC to PHY or PHY to MC), a set of signals and a family of timing parameters.

DFI 3.1 sub-interfaces

Sub-interface	Direction	Purpose
Control	MC -> PHY	Command, address, bank, chip select, CKE, ODT, reset
Write data	MC -> PHY	Write data, byte mask/DBI, write enable, write chip select
Read data	PHY -> MC	Read data, read valid, read chip select, DNV (LPDDR2)
Update	Both	MC-initiated and PHY-initiated update handshakes
Status	Both	Init, frequency ratio, clock disable, parity, alert
Training	Both	Gate training, read data eye training, write leveling, CA training
Low power control	MC -> PHY (ack PHY -> MC)	Low-power opportunity requests
Error	PHY -> MC	Error indication and error information

3.2 Clocking

The DFI clock is the MC clock. All DFI signals are referenced to the rising edge of this clock. The DFI PHY clock is the same frequency as the DRAM clock, which is one half of the data rate. In a matched-frequency system the MC clock and the PHY clock are the same; in a frequency-ratio system the PHY clock runs at a multiple of the MC clock.

DFI allows ratios of 1:1, 1:2 and 1:4. The ratio is conveyed on `dfi_freq_ratio` (2 bits): `00` for 1:1, `01` for 1:2, `10` for 1:4, `11` reserved.

3.3 Frequency ratio and the _pN / _wN replication rule

When the PHY clock is faster than the MC clock, the MC must supply multiple phases of command and data information per MC clock cycle. DFI 3.1 handles this with replicated signals:

- Commands and other MC-to-PHY signals that are sampled per PHY clock phase carry a `_pN` suffix. For example, `dfi_address_p0`, `dfi_address_p1`, `dfi_cs_n_p0`, `dfi_cs_n_p1`. The suffix for phase 0 is optional.
- Read data and read-data qualifiers from PHY to MC are delivered as DFI data words with a `_wN` suffix. For example, `dfi_rddata_w0`, `dfi_rddata_w1`, `dfi_rddata_valid_w0`, `dfi_rddata_valid_w1`.
- Variable-pulse-width status signals such as `dfi_alert_n` carry an `_aN` suffix to preserve pulse width across the clock-domain boundary.

The MC may issue commands on any phase. The PHY must be able to accept a command on any and all phases. For signals driven in all phases, such as `dfi_cke` and `dfi_odt`, the MC must drive every phase.

3.4 PHY data slices and byte lanes

DFI data width is generally twice the DRAM data width, because one DFI data word carries both edges of a DRAM data transfer. The PHY is often organized as data slices, each handling a byte or a small group of bytes.

`dfi_wrdata_en` and `dfi_rddata_en` are ideally one bit per PHY data slice, though the spec allows the width to differ between MC and PHY as long as the lowest bit maps to the lowest data segment. `dfi_rddata_valid` is one bit per PHY data slice so that each slice can return data independently.

`dfi_data_byte_disable` tells the PHY which byte lanes are unused. The bit order matches `dfi_wrdata` and `dfi_rddata`. This signal is defined at initialization and expected to remain constant.

3.5 Simplified teaching model

All timing examples and walkthroughs in this book use a deliberately small configuration so that phase and burst counts are obvious:

- One rank, one chip select.
- 1:2 frequency ratio: two PHY phases (`p0`, `p1`) per MC clock.
- DFI data width of 32 bits (two 16-bit PHY data words per MC clock), split into two slices.
- Burst length 8 on the DRAM, so a read or write transfers four DFI data words.
- `tphy_wrlat = 2`, `tphy_wrdata = 1`, `trddata_en = 2`, `tphy_rdlat = 4` unless stated otherwise.

Real systems are wider and may use 1:4 ratios; the scheduling rules are the same.

Source: DFI Specification v3.1 sections 2.0, 3.5.3, 4.8, 5.0

4 Acronyms

Acronyms used in this book

Term	Meaning
ACT	DRAM Activate command
BL	Burst length
CA	Command/Address bus (LPDDR3/LPDDR2)
CRC	Cyclic redundancy check
DBI	Data bus inversion
DFI	DDR PHY Interface
DFI PHY clock	PHY-side clock, equal to DRAM clock frequency
DRAM	Dynamic random-access memory
DQS	Data strobe
MC	Memory controller
MPR	Multi-purpose register (DRAM training mode)
MRR	Mode register read (LPDDR3/LPDDR2)
MRW	Mode register write
NOP	No operation command
ODT	On-die termination
PHY	Physical plane that drives DRAM pins
PRE	DRAM Precharge command
RD	DRAM Read command
REF	DRAM Refresh command
UI	Unit interval (half a data word)
WR	DRAM Write command

Suffix notation

DFI signal group	Meaning
_pN	Phase N of a frequency-ratio signal, MC to PHY
_wN	DFI data word N, PHY to MC in frequency-ratio systems
_aN	PHY clock cycle N for variable-pulse-width signals

Source: DFI Specification v3.1 section 6.0, Glossary

5 Control Interface

The control interface carries the command, address and control signals that the MC sends to the PHY. The PHY must preserve bit ordering and timing relationships so that, after its own `tctrl_delay`, the signals arrive at the DRAM pins correctly aligned.

5.1 Command/address signals

Control interface command and address signals

Signal	From	Width	Default	What it does
`dfi_address`	MC	DFI Address Width	none	Address bus. For DDR3 carries row/column address; for LPDDR3/LPDDR2 it maps the double-data-rate CA bus and must be at least 20 bits.
`dfi_bank`	MC	DFI Bank Width	none	Bank select bits.
`dfi_bg`	MC	DFI Bank Group Width	none	Bank group, DDR4 only. Unused for DDR3/LPDDR3.
`dfi_cid`	MC	DFI Chip ID Width	none	Stacked-device chip ID, DDR4 only. Unused for DDR3/LPDDR3.
`dfi_act_n`	MC	1 bit	none	DDR4 command encoding; held constant for LPDDR3/LPDDR2.

Signal	From	Width	Default	What it does
dfi_cas_n	MC	DFI Control Width	0x1	Column address strobe / command bit.
dfi_ras_n	MC	DFI Control Width	0x1	Row address strobe / command bit.
dfi_we_n	MC	DFI Control Width	0x1	Write enable / command bit.

In a frequency-ratio system these signals become dfi_address_pN, dfi_bank_pN, dfi_cas_n_pN, and so on. The MC may place a command on any phase; the PHY must accept commands on any and all phases.

5.2 DRAM enable and select signals

Control interface enable and select signals

Signal	From	Width	Default	What it does
`dfi_cs_n`	MC	DFI Chip Select Width	0x1	Chip select. Low active. For LPDDR3 CA training it carries the calibration command on the bit being trained.
`dfi_cke`	MC	DFI Chip Select Width	0x0 or 0x1	Clock enable. Must be driven in all phases. For LPDDR3 CA training it enables DRAM output drivers.
`dfi_odt`	MC	DFI Chip Select Width	0x0	On-die termination control. Used by DDR3, DDR2 and LPDDR3. Must be driven in all phases.
`dfi_reset_n`	MC	DFI Chip Select Width	0x0 or 0x1	DRAM reset. Used by DDR4 and DDR3.

5.3 LPDDR3/LPDDR2 CA-bus mapping

For LPDDR3 and LPDDR2, the DRAM CA bus is double data rate and is mapped onto the 20-bit `dfi_address` bus. The PHY transmits `dfi_address[9:0]` on the rising CA edge and `dfi_address[19:10]` on the falling CA edge. During these operations the signals `dfi_act_n`, `dfi_bank`, `dfi_bg`, `dfi_cid`, `dfi_cas_n`, `dfi_ras_n` and `dfi_we_n` must be held constant, and `dfi_address` must drive a NOP until `dfi_init_complete` asserts.

5.4 Control timing parameters

Control interface timing parameters

Parameter	Defined by	Meaning
`tcmd_lat`	MC	DFI PHY clock cycles from `dfi_cs_n` assertion to the associated CA signals being driven.
`tctrl_delay`	PHY	DFI clock cycles from a control signal change until that change reaches the DRAM interface.

`tctrl_delay` is what makes the DFI contract practical: the MC issues signals on its clock, and the PHY promises to deliver them to the DRAM after a fixed, rounded-up delay.

Source: DFI Specification v3.1 sections 3.1, 4.2, Table 4, Table 5

6 Write Data Interface

The write data interface carries write data from the MC to the PHY and tells the PHY when that data is valid. The PHY then retimes the data onto the DRAM DQ/DQS pins.

6.1 Write data signals

Write data interface signals

Signal	From	Width	Default	What it does
`dfi_wrdata`	MC	DFI Data Width	none	Write data bus. Data is driven `tphy_wrdata` cycles after `dfi_wrdata_en` asserts.
`dfi_wrdata_en`	MC	DFI Data Enable Width	0x0	Write data and mask enable. Asserted `tphy_wrlat` cycles after a write command; held for the duration of the data transfer.
`dfi_wrdata_mask`	MC	DFI Data Width / 8	none	Per-byte mask, or write DBI when DBI is enabled. Same timing as `dfi_wrdata`.
`dfi_wrdata_cs_n`	MC	CS Width x Data Enable Width	none	Optional write data chip select. Indicates the target rank for the data,

Signal	From	Width	Default	What it does
				and during write leveling indicates the rank being trained.

In a frequency-ratio system these signals are replicated as `dfi_wrdata_pN`, `dfi_wrdata_en_pN`, `dfi_wrdata_mask_pN` and `dfi_wrdata_cs_n_pN`. The `_pN` suffix gives the value for each PHY clock phase.

6.2 `dfi_wrdata_en` behavior

`dfi_wrdata_en` is the central enable for write data. One continuous assertion may cover data for multiple back-to-back write commands. The signal is asserted `tphy_wrlat` cycles after the write command on the control interface, and the actual data appears `tphy_wrdata` cycles after the enable. The enable de-asserts `tphy_wrdata` cycles before the last valid data word.

For a non-contiguous write, `dfi_wrdata_en` de-asserts between commands. For contiguous writes it stays asserted across command boundaries.

6.3 Write data mask and DBI

When DBI is disabled, `dfi_wrdata_mask[N]` masks the byte `dfi_wrdata[N*8+7:N*8]`. When DBI is enabled and `phydbi_mode = 0`, the same bus carries the write DBI inversion bits: 1 means the corresponding byte is inverted before being driven to DRAM. Mask and DBI are mutually exclusive.

6.4 Write chip select

`dfi_wrdata_cs_n` is optional in normal operation. When it is used, the PHY may need extra spacing (`tphy_wrcsgap`) between commands that target different chip selects so it can re-time its data path. The signal is driven `tphy_wrcslat` cycles after the write command and held for at least the data transfer length plus the gap time.

6.5 Write data timing parameters

Write data interface timing parameters

Parameter	Defined by	Meaning
`tphy_wrlat`	PHY	Cycles from write command to `dfi_wrdata_en` assertion.
`tphy_wrdata`	PHY	Cycles from `dfi_wrdata_en` assertion to `dfi_wrdata` driven.
`twrdata_delay`	System	Cycles from `dfi_wrdata_en` assertion to completion of the write data transfer on the DRAM bus.
`tphy_wrcslat`	PHY	Cycles from write command to `dfi_wrdata_cs_n` assertion.
`tphy_wrcsgap`	PHY	Additional cycles required between commands when changing target chip select.
`tphy_crcmin_lat`, `tphy_crcmax_lat`	System	Min/max cycles from CRC data on `dfi_wrdata_en` to CRC error on `dfi_alert_n`.

6.6 Write data programmable parameters

Write data programmable parameters

Parameter	Defined by	Meaning
`phycrc_mode`	PHY	`0` = MC generates CRC; `1` = PHY generates CRC.
`phydbi_mode`	PHY	`0` = MC generates DBI and inverts data; `1` = PHY does.

CRC and DBI are DDR4-era features and are not used by the scoria DDR3/LPDDR3 controller, but the signals and parameters are part of DFI 3.1 and are noted here for completeness.

Source: DFI Specification v3.1 sections 3.2, 4.4, Table 6, Table 7, Table 8

7 Read Data Interface

The read data interface returns captured DRAM data from the PHY to the MC. The MC tells the PHY how many words to expect; the PHY returns data when it is ready and marks each word valid.

7.1 Read data signals

Read data interface signals

Signal	From	Width	Default	What it does
dfi_rddata_en	MC	DFI Data Enable Width	0x0	Tells the PHY a read is underway and how many contiguous words are expected. Asserted trddata_en cycles after the read command.
dfi_rddata	PHY	DFI Data Width	none	Read data bus. Expected within tphy_rdlat cycles after dfi_rddata_en assertion.
dfi_rddata_valid	PHY	DFI Read Data Valid Width	0x0	One bit per PHY data slice, asserted with the correspondin g data.
dfi_rddata_dnv	PHY	DFI Data Width / 8	0x0	Data-not-valid indicator, per byte. Used by LPDDR2.
dfi_rddata_dbi_n	PHY	DFI DBI Width	none	Read DBI inversion bits.

Signal	From	Width	Default	What it does
				Used when `phydbi_mode = 0` and DBI is enabled.
`dfi_rddata_cs_n`	MC	CS Width x Data Enable Width	none	Optional read data chip select. During read training it indicates the rank being trained; in normal operation it identifies the target rank.

In a frequency-ratio system, `dfi_rddata_en` becomes `dfi_rddata_en_pN` while the return data and qualifiers become `dfi_rddata_wN`, `dfi_rddata_valid_wN`, `dfi_rddata_dnv_wN` and `dfi_rddata_dbi_n_wN`. The `_wN` suffix identifies the DFI data word number.

7.2 dfi_rddata_en and dfi_rddata_valid

dfi_rddata_en is the MC's prediction of how much read data is coming. It is asserted trddata_en cycles after the read command and held for the number of contiguous data words expected. The PHY responds with dfi_rddata_valid; there is a one-to-one correspondence between assertion cycles of dfi_rddata_en and dfi_rddata_valid.

tphy_rdlat is a maximum: the PHY may return data earlier than the programmed value. Each bit of dfi_rddata_valid corresponds to one PHY data slice, so different slices may return data on different cycles.

7.3 Read chip select

Like its write counterpart, `dfi_rddata_cs_n` is optional. When used, the PHY may require `tphy_rdcsgap` extra cycles between read commands that switch target chip selects. The signal is asserted `tphy_rdcslat` cycles after the read command.

7.4 LPDDR2 data-not-valid

LPDDR2 defines data words that are not all valid. In that case the PHY asserts `dfi_rddata_valid` for the returning word and simultaneously uses `dfi_rddata_dnv` to mark the invalid bytes. This is the only DRAM class in the v3.1 inventory that uses `dfi_rddata_dnv`.

7.5 Read data timing parameters

Read data interface timing parameters

Parameter	Defined by	Meaning
`trddata_en`	System	Cycles from read command to `dfi_rddata_en` assertion.
`tphy_rdlat`	PHY	Maximum cycles from `dfi_rddata_en` assertion to `dfi_rddata_valid` assertion.
`tphy_rdcslat`	PHY	Cycles from read command to `dfi_rddata_cs_n` assertion.
`tphy_rdcsgap`	PHY	Additional cycles required between commands when changing target chip select.

`trddata_en` and `tphy_rdlat` are normally held constant while commands are flowing; they may only be changed when the bus is idle.

Source: DFI Specification v3.1 sections 3.3, 4.5, Table 9, Table 10, Table 11

8 Update Interface

The update interface lets the PHY and MC pause the DFI bus to perform calibration, timing adjustments or other maintenance. The DFI bus must be idle during an update: no commands in flight, all read and write data finished, and write data fully retired on the DRAM bus.

8.1 MC-initiated update

MC-initiated update signals

Signal	From	Width	Default	What it does
`dfi_ctrlupd_req`	MC	1 bit	0x0	MC tells the PHY that the DFI bus will be idle and the PHY may update.
`dfi_ctrlupd_ack`	PHY	1 bit	0x0	Optional PHY acknowledge. If asserted it must be before `tctrlupd_min` expires.

`dfi_ctrlupd_req` must be asserted for at least `tctrlupd_min` cycles and no more than `tctrlupd_max` cycles. The maximum interval between assertions is `tctrlupd_interval`. The PHY is not required to acknowledge, but if it does the DFI bus stays idle while both request and acknowledge are asserted.

After self-refresh exit, `dfi_ctrlupd_req` must be asserted before read or write traffic resumes.

8.2 PHY-initiated update

PHY-initiated update signals

Signal	From	Width	Default	What it does
dfi_phyupd_req	PHY	1 bit	0x0	PHY requests idle time on the DFI bus.
dfi_phyupd_type	PHY	2 bits	none	Selects one of four update types.
dfi_phyupd_ack	MC	1 bit	0x0	MC grants the request.

When the PHY needs the bus idle, it asserts dfi_phyupd_req with a valid dfi_phyupd_type. The MC must respond with dfi_phyupd_ack within tphyupd_resp cycles. The request remains asserted until the update completes, and must de-assert before tphyupd_typeX cycles have elapsed after the acknowledge. The MC must not assert dfi_ctrlupd_req and dfi_phyupd_ack at the same time.

8.3 Update timing parameters

Update interface timing parameters

Parameter	Defined by	Meaning
tctrlupd_min	MC	Minimum cycles dfi_ctrlupd_req must be asserted.
tctrlupd_max	MC	Maximum cycles dfi_ctrlupd_req may be asserted.
tctrlupd_interval	MC	Maximum cycles the MC may wait between dfi_ctrlupd_req assertions.
tphyupd_type0	PHY	Max cycles for dfi_phyupd_type = 0.
tphyupd_type1	PHY	Max cycles for dfi_phyupd_type = 1.
tphyupd_type2	PHY	Max cycles for dfi_phyupd_type = 2.
tphyupd_type3	PHY	Max cycles for dfi_phyupd_type = 3.
tphyupd_resp	PHY	Max cycles from dfi_phyupd_req to dfi_phyupd_ack.

8.4 Simultaneous requests

If `dfi_ctrlupd_req` and `dfi_phyupd_req` are asserted together, the MC is forbidden from asserting `dfi_phyupd_ack` at the same time as `dfi_ctrlupd_req`. The PHY may de-assert its own request in this case, and the acknowledged request follows its normal protocol.

Source: DFI Specification v3.1 sections 3.4, 4.6, Table 12, Table 13

9 Status Interface

The status interface conveys configuration and status information between the MC and the PHY. It covers initialization, frequency ratio, DRAM clock disable, command parity and the alert signal.

9.1 Initialization and configuration signals

Status interface initialization and configuration signals

Signal	From	Width	Default	What it does
`dfi_init_start`	MC	1 bit	0x0 or 0x1	At init, indicates `dfi_data_byte_disable` and `dfi_freq_ratio` are valid. During normal operation, requests a frequency change.
`dfi_init_complete`	PHY	1 bit	0x0	PHY is ready for normal DFI transactions. All command/status signals must be at default until this asserts.
`dfi_data_byte_disable`	MC	DFI Data Width / 8	none	Marks unused byte lanes so the PHY may power them down. Defined at initialization only.
`dfi_freq_ratio`	MC	2 bits	none	`00`=1:1, `01`=1:2, `10`=1:4,

Signal	From	Width	Default	What it does
				11=reserved. Required only for devices supporting multiple ratios.
`dfi_dram_clk_disable`	MC	DFI Chip Select Width	0x0	When asserted the PHY holds DRAM clocks at a constant value.

Initialization completes when both `dfi_init_start` and `dfi_init_complete` are asserted simultaneously for at least one DFI clock cycle. The PHY may assert `dfi_init_complete` before `dfi_init_start` if it does not need the configuration signals.

9.2 Parity and alert signals

Status interface parity and alert signals

Signal	From	Width	Default	What it does
dfi_parity_in	MC	1 bit	0x0 or 0x1	Command parity bit, one per command. 0=even, 1=odd over the covered CA signals. Phase-indexed in frequency-ratio systems.
dfi_alert_n_aN	PHY	DFI Alert Width	1	Active-low CRC or command-parity error indicator propagated from DRAM ALERT_N. Pulse width preserved with _aN suffix.

dfi_parity_in covers dfi_address, dfi_bank, dfi_bg, dfi_act_n, dfi_cas_n, dfi_ras_n and dfi_we_n. It is valid for one cycle per command and is delayed by tparin_lat relative to the command. The PHY delays it by the same tctrl_delay as the command bus. dfi_alert_n asserts within tphy_paritylat cycles of the associated parity bit.

These signals are DDR4/DDR3 CA-parity and CRC features and are not used by the scoria DDR3/LPDDR3 controller.

9.3 Status timing parameters

Status interface timing parameters

Parameter	Defined by	Meaning
`tinit_start`	MC	Max cycles from `dfi_init_start` assertion to PHY de-asserting `dfi_init_complete` for a frequency change.
`tinit_complete`	PHY	Max cycles from `dfi_init_start` de-assertion to `dfi_init_complete` re-assertion after re-initialization.
`tdram_clk_disable`	PHY	Cycles from `dfi_dram_clk_disable` assertion to DRAM clock held constant.
`tdram_clk_enable`	PHY	Cycles from `dfi_dram_clk_disable` de-assertion to first valid DRAM clock edge.
`tparin_lat`	MC	Cycles from command to associated `dfi_parity_in`.
`tphy_paritylat`	PHY	Max cycles from `dfi_parity_in` to `dfi_alert_n` assertion.

Source: DFI Specification v3.1 sections 3.5, 4.1, 4.7, 4.9, 4.10, Table 14, Table 15

10 Training Interface

DFI 3.1 defines four training operations: gate training, read data eye training, write leveling and CA training. Gate training and read data eye training together are called “read training”. The DRAM class determines which operations a system uses.

Training operations in DFI 3.1

Operation	DRAM classes	Purpose
Gate training	DDR4, DDR3, LPDDR3, LPDDR2	Center the read DQS gate in the read preamble.
Read data eye training	DDR4, DDR3, LPDDR3, LPDDR2	Center read DQS in the DQ data eye.
Write leveling	DDR4, DDR3, LPDDR3	Align write DQS to the DRAM clock.
CA training	LPDDR3 only	Optimize CA bus setup/hold relative to the memory clock.

10.1 Read training signals

Read training signals

Signal	From	Width	Default	What it does
dfi_rdlvl_en	MC	DFI Read Leveling MC I/F Width	0x0	Enables read data eye training logic in the PHY.
dfi_rdlvl_req	PHY	DFI Read Leveling PHY I/F Width	0x0	PHY requests read data eye training; MC must respond with dfi_rdlvl_en within trdlvl_resp cycles.
dfi_rdlvl_gate_en	MC	DFI Read Leveling MC I/F Width	0x0	Enables gate training logic in the PHY.
dfi_rdlvl_gate_req	PHY	DFI Read Leveling PHY I/F Width	0x0	PHY requests gate training; MC must respond with dfi_rdlvl_gate_en within trdlvl_resp cycles.
dfi_rdlvl_resp	PHY	DFI Read Leveling Response Width	none	Per-slice indication that training is complete.
dfi_phy_rdlvl_cs_n	PHY	CS Width x Read Training PHY I/F Width	0x1	Target chip select for read data eye

Signal	From	Width	Default	What it does
				training.
`dfi_phy_rdlvl_gate_cs_n`	PHY	CS Width x Read Training PHY I/F Width	0x1	Target chip select for gate training.

When either read-training enable de-asserts, both the MC and the PHY must reset their DFI read data word pointers to zero.

10.2 Write leveling signals

Write leveling signals

Signal	From	Width	Default	What it does
`dfi_wrlvl_en`	MC	DFI Write Leveling MC I/F Width	0x0	Enables write leveling logic in the PHY.
`dfi_wrlvl_req`	PHY	DFI Write Leveling PHY I/F Width	0x0	PHY requests write leveling; MC must respond with `dfi_wrlvl_en` within `twrlvl_resp` cycles.
`dfi_wrlvl_strobe`	MC	DFI Write Leveling MC I/F Width	0x0	One-DFI-clock pulse that tells the PHY to capture the write leveling response from DQ.
`dfi_wrlvl_resp`	PHY	DFI Write Leveling Response Width	none	Per-slice indication that write leveling is complete.
`dfi_phy_wrlvl_cs_n`	PHY	CS Width x Write Leveling PHY I/F Width	0x1	Target chip select for write leveling.

Write leveling is the operation that DDR3 adds over DDR2, and it is the main reason scoria moved from a v2.1.1-style interface to v3.1.

10.3 CA training signals (LPDDR3)

CA training signals

Signal	From	Width	Default	What it does
`dfi_calvl_en`	MC	DFI CA Training MC I/F Width	none	Enables LPDDR3 CA training logic. Also acknowledges `dfi_calvl_req`.
`dfi_calvl_req`	PHY	DFI CA Training PHY I/F Width	none	Optional PHY request for CA training.
`dfi_calvl_capture`	MC	DFI CA Training MC I/F Width	none	One-cycle pulse telling the PHY to capture the CA value from DQ.
`dfi_calvl_resp`	PHY	DFI CA Training Response Width	none	`00`=not done, `01`=done/do not change segment, `10`=done/change segment, `11`=complete.
`dfi_phy_calvl_cs_n`	PHY	DFI Chip Select Width	none	Target chip select for CA training.

During CA training the control signals `dfi_address`, `dfi_cke` and `dfi_cs_n` take on training-specific roles: `dfi_address` carries background and calibration patterns, `dfi_cke` enables DRAM output drivers, and `dfi_cs_n` becomes the calibration command.

10.4 PHY-requested training in non-DFI training mode

PHY-requested training signals

Signal	From	Width	Default	What it does
`dfi_phylvl_req_cs_n`	PHY	DFI Rank Width	none	PHY requests control of the DRAM bus to train one chip select.
`dfi_phylvl_ack_cs_n`	MC	DFI Chip Select Width	none	MC grants the request when the bus is idle and all pages are closed.

This interface, new in v3.1, lets the PHY ask for the DRAM bus during normal operation so it can perform its own training without using the DFI training sequences. The MC idles the bus and grants the request.

10.5 Common training controls

Common training control signals

Signal	From	Width	Default	What it does
`dfi_lvl_pattern`	MC	4 bits (DDR4) or 1 bit (LPDDR3/LPDDR2)	0x0 / 0x1	Selects the training pattern for gate/read data eye training.
`dfi_lvl_periodic`	MC	DFI Leveling PHY I/F Width	0	`0`=full/long training; `1`=periodic/short training.

10.6 Training timing parameters

Training interface timing parameters

Parameter	Defined by	Meaning
`trdlvl_en`	PHY	Minimum cycles from read training enable to first read command.
`trdlvl_resp`	MC	Max cycles from `dfi_rdlvl_req`/`dfi_rdlvl_gate_req` to enable assertion.
`trdlvl_max`	MC	Max cycles the MC waits for `dfi_rdlvl_resp`.
`trdlvl_rr`	PHY	Minimum cycles between read training reads.
`twrlvl_en`	PHY	Minimum cycles from write leveling enable to first `dfi_wrlvl_strobe`.
`twrlvl_resp`	MC	Max cycles from `dfi_wrlvl_req` to `dfi_wrlvl_en` assertion.
`twrlvl_max`	MC	Max cycles the MC waits for `dfi_wrlvl_resp`.
`twrlvl_ww`	PHY	Minimum cycles between `dfi_wrlvl_strobe` assertions.
`tcalvl_en`	PHY	Minimum cycles from `dfi_calvl_en` to `dfi_cke` de-assertion.
`tcalvl_capture`	PHY	Cycles from calibration command to `dfi_calvl_capture`

Parameter	Defined by	Meaning
		pulse.
tcalvl_resp	MC	Max cycles from dfi_calvl_req to dfi_calvl_en.
tcalvl_max	MC	Max cycles the MC waits for dfi_calvl_resp.
tcalvl_cc	PHY	Minimum cycles between calibration commands.
tphylvl	PHY	Max cycles dfi_phylvl_req_cs_n may stay asserted after acknowledge.
tphylvl_resp	PHY	Max cycles from request to acknowledge for PHY-requested training.

10.7 Training programmable parameters

Training programmable parameters

Parameter	Defined by	Meaning
`phyrdlvl_en`	PHY	PHY supports read data eye training.
`phyrdlvl_gate_en`	PHY	PHY supports gate training.
`phywrlvl_en`	PHY	PHY supports write leveling.
`phycalvl_en`	PHY	PHY supports CA training.

Source: DFI Specification v3.1 sections 3.6, 4.11, Table 16, Table 17, Table 18

11 Low Power Control Interface

The low power control interface lets the MC inform the PHY of an opportunity to enter a low-power state. DFI 3.1 splits the single v2.1 `dfi_lp_req` into two requests: one for the control path and one for the data path. This lets the MC and PHY sleep the paths independently.

11.1 Low power signals

Low power control interface signals

Signal	From	Width	Default	What it does
`dfi_lp_ctrl_req`	MC	1 bit	0x0	MC indicates no more commands will be sent on the control interface.
`dfi_lp_data_req`	MC	1 bit	0x0	MC indicates no more commands will be sent on the data interface.
`dfi_lp_ack`	PHY	1 bit	0x0	Optional PHY acknowledge.
`dfi_lp_wakeup`	MC	4 bits	none	Encoded wakeup time. Valid only while a request is asserted.

Only one low-power request may be outstanding until it is acknowledged or aborted. When exiting, if both requests are asserted they must de-assert simultaneously.

11.2 dfi_lp_wakeup encoding

Low power wakeup encoding

dfi_lp_wakeup	Wakeup time (DFI clock cycles)
0000	16
0001	32
0010	64
0011	128
0100	256
0101	512
0110	1024
0111	2048
1000	4096
1001	8192
1010	16384
1011	32768
1100	65536
1101	131072
1110	262144
1111	unlimited

The MC must hold `dfi_lp_wakeup` constant until `dfi_lp_ack` asserts. After acknowledge it may increase the value but never decrease it. The value at the moment the request de-asserts defines the `tlp_wakeup` time.

11.3 Low power handshake

1. The MC asserts `dfi_lp_ctrl_req` or `dfi_lp_data_req` with a static `dfi_lp_wakeup` value.
2. The PHY may assert `dfi_lp_ack` within `tlp_resp` cycles.
3. While request and acknowledge are both asserted, the PHY may remain in a low-power state.
4. When the request de-asserts, the PHY must return to normal operation and de-assert `dfi_lp_ack` within `tlp_wakeup` cycles.

If the PHY does not acknowledge within `tlp_resp` cycles, the MC may de- assert the request and treat the opportunity as declined.

11.4 Low power timing parameters

Low power control interface timing parameters

Parameter	Defined by	Meaning
`tlp_resp`	MC	Max cycles from request assertion to `dfi_lp_ack` assertion.
`tlp_wakeup`	MC	Max cycles from request de-assertion to `dfi_lp_ack` de-assertion.

In the scoria DDR3/LPDDR3 controller this interface is exposed for spec compliance but is not consumed by the target PHY family.

Source: DFI Specification v3.1 sections 3.7, 4.12, Table 19, Table 20

12 Error Interface

The error interface lets the PHY report error conditions to the MC. It is optional for both MC and PHY, and is not phased for frequency-ratio systems.

12.1 Error signals

Error interface signals

Signal	From	Width	Default	What it does
`dfi_error`	PHY	DFI Error Width	0	Indicates the PHY has detected an error. Typically one bit per data slice plus one bit for control.
`dfi_error_info`	PHY	DFI Error Width x 4	0	Additional information about the error source. Valid only while `dfi_error` is asserted.

The PHY may implement `dfi_error` as a single bit or as one bit per data slice plus one control bit. `dfi_error_info` provides four bits per error instance. The specification defines only a few general encodings; the rest are design-specific. Undriven bits must be tied low at the MC.

12.2 Error timing

Error interface timing parameter

Parameter	Defined by	Meaning
`terror_resp`	PHY	Max cycles from the affected transaction to `dfi_error` assertion.

For data errors the reference point is `dfi_wrdata_en` or `dfi_rddata_en`. For command errors the reference point is the command itself. Because `terror_resp` is a maximum, the MC may not be able to correlate the error with a single transaction.

12.3 Spec-defined error codes

DFI 3.1 error codes

dfi_error_info	Name	Meaning
0000	General purpose	PHY general-purpose error.
0001	Internal PHY	PHY-specific internal error.
0010-0111	Reserved	Reserved for future definition.
1000-1111	User-defined	Available for design-specific codes.

The scoria controller uses the error indication but ignores the DDR4- specific encodings, treating `dfi_error` as a generic PHY fault flag.

Source: DFI Specification v3.1 sections 3.8, 4.13, Table 21, Table 22, Table 27

13 Reset and Initialization

DFI 3.1 does not dictate a power-up sequence for either the MC or the PHY, but it does define the handshake that marks the boundary between initialization and normal operation.

13.1 Default values before `dfi_init_complete`

Until `dfi_init_complete` asserts, every command and status signal must be held at its default value. The defaults come from the interface tables:

- High-active signals default to `0` (for example `dfi_odt`, `dfi_wrdata_en`).
- Low-active signals default to `1` (for example `dfi_cs_n`, `dfi_we_n`).
- Buses have no specified default.
- State-specific signals such as `dfi_dram_clk_disable` and `dfi_parity_in` take their system-defined values.

For LPDDR3 and LPDDR2 systems, `dfi_address` must drive a NOP command until `dfi_init_complete` asserts.

13.2 The dfi_init_start / dfi_init_complete handshake

dfi_init_start has two functions:

1. At initialization it tells the PHY that dfi_data_byte_disable and dfi_freq_ratio are valid. The PHY may wait for this assertion before asserting dfi_init_complete if it needs those values.
2. During normal operation it requests a frequency change.

Initialization completes when both dfi_init_start and dfi_init_complete are asserted for at least one DFI clock cycle. After that, the MC may hold or de-assert dfi_init_start.

Initialization handshake states

Phase	dfi_init_start	dfi_init_complete	Meaning
Power-up	driven by MC	0	PHY not ready.
Config valid	1	0	MC has defined ratio and byte disable.
Complete	1	1 for >=1 cycle	PHY ready; normal operation may begin.

13.3 Frequency-change reuse of the init signals

During normal operation the MC may assert `dfi_init_start` again to request a frequency change. The PHY must de-assert `dfi_init_complete` within `tinit_start` cycles to acknowledge the request. If it does not, the MC must abort and release `dfi_init_start`.

Once acknowledged, the MC holds `dfi_init_start` asserted while the clock frequency changes. When the change is done, the MC de-asserts `dfi_init_start`; the PHY then re-initializes and re-asserts `dfi_init_complete` within `tinit_complete` cycles.

The DFI bus must be kept at valid and stable levels throughout the frequency change. Both sides reset their DFI read data word pointers to zero when `dfi_init_start` asserts.

13.4 Initialization timing parameters

Initialization timing parameters

Parameter	Defined by	Meaning
`tinit_start`	MC	Max cycles for the PHY to acknowledge a frequency-change request.
`tinit_complete`	PHY	Max cycles for the PHY to re-assert `dfi_init_complete` after `dfi_init_start` de-asserts.

Source: DFI Specification v3.1 sections 3.5.1, 4.1, 4.9, Figure 3, Figure 4, Figure 5

14 Training in DFI 3.1

DFI 3.1 supports four training operations. A system does not have to use them all; the DRAM class and the PHY capabilities determine which are required.

14.1 Which training is used where

Training operations by memory type

Training	DDR3	LPDDR3	Notes
Gate training	Yes	Yes	Centers read DQS gate in preamble.
Read data eye training	Yes	Yes	Centers read DQS in DQ eye.
Write leveling	Yes	Yes	Aligns write DQS to DRAM clock.
CA training	No	Yes	LPDDR3 only; optimizes CA timing.

For DFI compliance the MC must support the training operations that apply to the DRAM types it supports. The PHY may optionally support each operation.

14.2 DFI training mode

In DFI training mode the MC sets the DRAM into the appropriate training mode, issues the necessary read commands or write strobes, and lets the PHY adjust its own delays. The PHY signals completion by asserting `dfi_rdlvl_resp`, `dfi_wrlvl_resp` or `dfi_calvl_resp`. The MC then de- asserts the enable and returns to normal operation.

The MC must finish all normal traffic before starting training. During training the only timing parameters that matter are:

- `trdlvl_rr`: minimum spacing between read training reads.
- `twrlvl_ww`: minimum spacing between write leveling strobes.
- `tcalvl_cc`: minimum spacing between CA calibration commands.

14.3 Read training sequence

1. The PHY asserts `dfi_rdlvl_req` (data eye) or `dfi_rdlvl_gate_req` (gate). It may indicate the target chip select on `dfi_phy_rdlvl_cs_n` or `dfi_phy_rdlvl_gate_cs_n`.
2. The MC responds with `dfi_rdlvl_en` or `dfi_rdlvl_gate_en` within `trdlvl_resp` cycles.
3. The MC issues training reads at least `trdlvl_rr` cycles apart. For DDR3 these are normal reads; for LPDDR3 they are mode-register reads.
4. The PHY completes its delay search and asserts `dfi_rdlvl_resp`.
5. The MC de-asserts the enable.

14.4 Write leveling sequence

1. The PHY asserts `dfi_wrlvl_req`, optionally with `dfi_phy_wrlvl_cs_n`.
2. The MC responds with `dfi_wrlvl_en` within `twrlvl_resp` cycles.
3. The MC issues MRS commands to enter and exit write leveling mode in the DRAM.
4. The MC issues `dfi_wrlvl_strobe` pulses at least `twrlvl_ww` cycles apart; the PHY captures the DQ response after each strobe.
5. The PHY asserts `dfi_wrlvl_resp` when complete.
6. The MC de-asserts `dfi_wrlvl_en`.

14.5 CA training sequence (LPDDR3)

1. The PHY may assert `dfi_calvl_req` with `dfi_phy_calvl_cs_n`.
2. The MC responds with `dfi_calvl_en` within `tcalvl_resp` cycles.
3. The MC drives calibration patterns on `dfi_address` and calibration commands on `dfi_cs_n`, with `dfi_cke` used to enable DRAM outputs.
4. The MC asserts `dfi_calvl_capture` `tcalvl_capture` cycles after a calibration command so the PHY captures the returned DQ value.
5. The PHY reports progress on `dfi_calvl_resp`.
6. The MC de-asserts `dfi_calvl_en` when `dfi_calvl_resp` indicates completion.

14.6 Periodic vs full training

`dfi_lvl_periodic` selects the training length:

- `0`: full training, used at initialization, self-refresh restart or after large delay changes.
- `1`: periodic training, a shorter tuning pass during normal operation.

14.7 Non-DFI training mode and PHY-requested training

If the PHY supports non-DFI training, it may request the DRAM bus with `dfi_phylvl_req_cs_n`. The MC grants it with `dfi_phylvl_ack_cs_n` only when the bus is idle and all pages are closed. The PHY then trains on its own and returns control when done. This is new in v3.1 and is separate from the DFI training sequences above.

Source: DFI Specification v3.1 sections 3.6, 4.11, Table 16, Table 17, Table 18

15 Command Mapping

On the DFI control interface, DRAM commands are encoded by the command signals `dfi_act_n`, `dfi_ras_n`, `dfi_cas_n` and `dfi_we_n`, together with the address, bank and chip-select buses. The PHY forwards these values to the DRAM pins with the timing controlled by `tcmd_lat` and `tctrl_delay`.

15.1 DDR3 command encoding on DFI

For DDR3 the command mapping is the familiar JEDEC encoding:

DDR3 command encoding on DFI control signals

Command	dfi_act_n	dfi_ras_n	dfi_cas_n	dfi_we_n
Activate (ACT)	1	0	1	1
Read (RD)	1	1	0	1
Write (WR)	1	1	0	0
Precharge (PRE)	1	0	1	0
Refresh (REF)	1	0	0	1
Mode Register Set (MRS)	1	0	0	0
NOP	1	1	1	1
Deselect	1	1	1	0

`dfi_act_n` is held high for DDR3; it is only meaningful for DDR4, where an active-low `dfi_act_n` repurposes `dfi_ras_n`, `dfi_cas_n` and `dfi_we_n` as upper row address bits A16, A15 and A14.

15.2 LPDDR3/LPDDR2 CA-bus mapping

For LPDDR3 and LPDDR2 the DRAM has no separate RAS/CAS/WE pins. Instead the command, address and bank information travel on a double-data-rate CA bus. DFI maps this CA bus onto the `dfi_address` bus:

LPDDR3/LPDDR2 CA mapping onto `dfi_address`

`dfi_address` bit	CA rising edge bit	CA falling edge bit
9:0	CA9:CA0	-
19:10	-	CA9:CA0

The PHY transmits `dfi_address[9:0]` on the rising CA edge and `dfi_address[19:10]` on the falling CA edge. During LPDDR3/LPDDR2 operation the command signals `dfi_ras_n`, `dfi_cas_n`, `dfi_we_n`, `dfi_act_n`, `dfi_bank`, `dfi_bg` and `dfi_cid` must be held constant.

15.3 Chip select and enable usage

`dfi_cs_n` is low active and selects the target rank. `dfi_cke` enables the DRAM clock. `dfi_odt` controls on-die termination and is relevant for DDR3, DDR2 and LPDDR3. All three are driven in every phase in a frequency- ratio system.

`dfi_reset_n` is used by DDR4 and DDR3. The scoria controller drives it from its init sequencer and presents it as `dfi_reset_n_o` at the DFI boundary.

15.4 Command timing relationship

The MC drives `dfi_cs_n` and the command/address signals according to `tcmd_lat`. If `tcmd_lat` is non-zero, the CA signals are driven that many DFI PHY clock cycles after `dfi_cs_n` assertion. The PHY then applies its own `tctrl_delay` before driving the DRAM pins.

Source: DFI Specification v3.1 sections 3.1, 4.2, Table 4

16 Write Carriage

A DFI write transaction has two separable delays: the time from the write command to the write-data enable, and the time from the enable to the data itself. The PHY defines both delays; the MC schedules around them.

16.1 Write timing parameters in use

Write carriage timing parameters

Parameter	Meaning
`tphy_wrlat`	Cycles from write command to `dfi_wrdata_en` assertion.
`tphy_wrdata`	Cycles from `dfi_wrdata_en` assertion to `dfi_wrdata` driven.
`twrdata_delay`	Cycles from `dfi_wrdata_en` assertion to completion on the DRAM bus.

Both `tphy_wrlat` and `tphy_wrdata` can be zero. They must be held constant while commands are flowing; they may only be changed when the bus is idle.

16.2 Single write transaction

1. The MC issues a write command on the control interface.
2. `tphy_wrlat` cycles later, the MC asserts `dfi_wrdata_en`.
3. `tphy_wrdata` cycles after that, the MC drives the first `dfi_wrdata` word.
4. The MC holds `dfi_wrdata_en` for the length of the transfer. For a burst-length-8 write this is four DFI data words.
5. `dfi_wrdata_en` de-asserts `tphy_wrdata` cycles before the last valid data word.
6. The write completes on the DRAM bus `twrdata_delay` cycles after `dfi_wrdata_en` first asserted.

16.3 Contiguous vs non-contiguous writes

For back-to-back writes with matching timing, `dfi_wrdata_en` can stay asserted across command boundaries and the data stream is contiguous. If the command spacing is larger or the timing parameters differ, the enable and data de-assert between commands.

Write data stream patterns

Case	`dfi_wrdata_en`	Data stream
Back-to-back, same parameters	Continuous	Contiguous
Interrupted first burst	Asserted for partial first burst plus full second burst	Contiguous for part of first plus all of second
Independent writes	De-asserts between commands	Non-contiguous

16.4 Write chip select timing

When `dfi_wrdata_cs_n` is used, the PHY defines `tphy_wrcslat` and `tphy_wrcsgap`. The MC drives the chip select `tphy_wrcslat` cycles after the write command and holds it for the data transfer length plus the gap time. If the next write targets a different chip select, the MC must add `tphy_wrcsgap` cycles between commands.

16.5 DBI and CRC notes

When DBI is enabled and `phydbi_mode = 0`, `dfi_wrdata_mask` carries the per-byte inversion bits instead of mask bits. When CRC is enabled and `phycrc_mode = 0`, the MC sends the CRC data word on the `dfi_wrdata` bus and extends `dfi_wrdata_en` accordingly. These are DDR4 features and are not used by the scoria DDR3/LPDDR3 controller.

Source: DFI Specification v3.1 sections 3.2, 4.4, Table 6, Table 7

17 Read Carriage

A DFI read transaction is split into the MC's request (`dfi_rddata_en`) and the PHY's response (`dfi_rddata` plus `dfi_rddata_valid`). The MC knows how many words to expect; the PHY returns them when ready.

17.1 Read timing parameters in use

Read carriage timing parameters

Parameter	Meaning
`trddata_en`	Cycles from read command to `dfi_rddata_en` assertion.
`tphy_rdlat`	Maximum cycles from `dfi_rddata_en` assertion to `dfi_rddata_valid` assertion.

`trddata_en` and `tphy_rdlat` are normally held constant while commands flow. They may only be changed when the bus is idle.

17.2 Single read transaction

1. The MC issues a read command on the control interface.
2. `trddata_en` cycles later, the MC asserts `dfi_rddata_en` for the expected number of data words. For burst-length-8 this is four DFI data words.
3. Within `tphy_rdlat` cycles, the PHY asserts `dfi_rddata_valid` and drives the first `dfi_rddata` word.
4. The PHY holds `dfi_rddata_valid` for the same number of cycles that `dfi_rddata_en` was asserted.
5. The MC de-asserts `dfi_rddata_en` once the expected data has returned.

17.3 Contiguous vs non-contiguous reads

Like writes, reads may be contiguous or separated by bubbles. The `dfi_rddata_en` signal can stay asserted across multiple read commands if no gap is expected; `dfi_rddata_valid` follows with the actual data. The PHY may return data earlier than the maximum `tphy_rdlat`, so the actual valid pattern may vary even when the enable is contiguous.

17.4 Per-slice independence

`dfi_rddata_valid` is one bit per PHY data slice. Different slices may return data on different cycles, and `tphy_rdlat` is the maximum across all slices. This is important for wide data buses where read capture timing varies by slice.

17.5 Read chip select timing

When `dfi_rddata_cs_n` is used, the MC drives it `tphy_rdcslat` cycles after the read command and holds it for the transfer length plus `tphy_rdcsgap`. If the next read targets a different chip select, the MC adds `tphy_rdcsgap` cycles.

17.6 LPDDR2/LPDDR3 mode-register reads

For LPDDR3 and LPDDR2, mode-register reads (MRR) use the same DFI read interface. The MC issues the MRR as a read command, asserts `dfi_rddata_en` for the expected return length, and the PHY returns the mode-register value on `dfi_rddata`. The `dfi_rddata_dnv` signal marks any invalid bytes in the returned word.

Source: DFI Specification v3.1 sections 3.3, 4.5, Table 9, Table 10

18 Walkthroughs

These walkthroughs use a deliberately tiny teaching model so that every DFI wire can be traced by hand.

18.1 Teaching model

- One rank, one chip select (`dfi_cs_n` is a single bit).
- 1:2 frequency ratio: two PHY phases (`p0`, `p1`) per MC clock.
- DFI data width 32 bits, split into two 16-bit PHY data slices.
- Burst length 8 on DRAM, so each read or write moves four DFI data words.
- Fixed timing: `tcmd_lat = 0`, `tctrl_delay = 1`, `tphy_wrlat = 2`, `tphy_wrdata = 1`, `trddata_en = 2`, `tphy_rdlat = 4`.

18.2 Walkthrough 1: activate -> read -> data return

The controller wants to read from bank 0, row 5, column 3.

Cycle 0: - Control interface: `dfi_cs_n = 0`, `dfi_ras_n = 0`, `dfi_cas_n = 1`, `dfi_we_n = 1`, `dfi_bank = 0`, `dfi_address = row 5`. This is an ACT. - `dfi_cke = 1`, `dfi_odt` driven to its programmed value.

Cycle 1: - Control interface: `dfi_cs_n = 1` (NOP / deselect).

Cycle 4: - After `tRCD` DRAM timing has been satisfied, the MC issues a read. - Control interface: `dfi_cs_n = 0`, `dfi_ras_n = 1`, `dfi_cas_n = 0`, `dfi_we_n = 1`, `dfi_bank = 0`, `dfi_address = column 3`.

Cycle 6: - `trddata_en = 2` cycles after the read, the MC asserts `dfi_rddata_en` for four cycles to request four DFI data words.

Cycle 10: - `tphy_rdlat = 4` cycles after `dfi_rddata_en`, the PHY asserts `dfi_rddata_valid` and drives the first `dfi_rddata` word.

Cycles 10-13: - The PHY returns four consecutive data words on `dfi_rddata`, with `dfi_rddata_valid` asserted each cycle.

18.3 Walkthrough 2: activate -> write with preamble and mask

The controller wants to write `0xA5A5A5A5` to bank 1, row 2, column 7, masking the lowest byte.

Cycle 0: - Control interface: `dfi_cs_n = 0`, `dfi_ras_n = 0`, `dfi_cas_n = 1`, `dfi_we_n = 1`, `dfi_bank = 1`, `dfi_address = row 2`. ACT.

Cycle 4: - After row-open timing, the MC issues a write. - Control interface: `dfi_cs_n = 0`, `dfi_ras_n = 1`, `dfi_cas_n = 0`, `dfi_we_n = 0`, `dfi_bank = 1`, `dfi_address = column 7`.

Cycle 6: - `tphy_wrlat = 2` cycles after the write command, the MC asserts `dfi_wrdata_en` for four cycles.

Cycle 7: - `tphy_wrdata = 1` cycle after the enable, the MC drives the first `dfi_wrdata` word: `0xA5A5A5A5`. - `dfi_wrdata_mask` is driven as `0x01` to mask the lowest byte. Bit 0 masks the byte in `dfi_wrdata[7:0]`.

Cycles 7-10: - The MC continues driving the same data word and mask for all four cycles of the burst.

Cycle 11: - `dfi_wrdata_en` has de-asserted one cycle before the last data word. - The write data completes on the DRAM bus after `twrdata_delay` cycles.

18.4 Walkthrough 3: back-to-back reads on different rows

Bank 0 row 5 and bank 1 row 2 are both opened by earlier activates. The controller issues two reads, one to each bank.

Cycle 0: - Read from bank 0, column 3: `dfi_cs_n = 0`, `dfi_cas_n = 0`, `dfi_we_n = 1`, `dfi_bank = 0`, `dfi_address = column 3`.

Cycle 2: - `dfi_rddata_en` asserts for four cycles (bank 0 read data expected).

Cycle 4: - Second read from bank 1, column 7: `dfi_cs_n = 0`, `dfi_cas_n = 0`, `dfi_we_n = 1`, `dfi_bank = 1`, `dfi_address = column 7`. - Because the two reads are contiguous, `dfi_rddata_en` stays asserted for four more cycles.

Cycle 6: - `dfi_rddata_valid` asserts for the first read data (earliest allowed by `tphy_rdlat = 4` from cycle 2).

Cycle 10: - `dfi_rddata_valid` asserts for the second read data (four cycles after the second `dfi_rddata_en` assertion at cycle 6).

The data stream on `dfi_rddata` is contiguous even though the two reads went to different banks, because the controller kept `dfi_rddata_en` asserted across both commands.

Source: DFI Specification v3.1 sections 4.4, 4.5

19 Parameter Families

DFI timing parameters are named `dfi_t*` in the specification text, though the signal names omit the `dfi_` prefix for brevity. Each family constrains a different part of the MC/PHY contract.

19.1 PHY-defined vs MC/system-defined parameters

Timing parameter ownership

Defined by	Examples	What the value tells you
PHY	`tctrl_delay`, `tphy_wrlat`, `tphy_wrdata`, `tphy_rdlat`, `tphy_rdcslat`, `tphy_wrcslat`, `tphy_*_gap`	How much time the PHY needs internally to retime signals.
MC/system	`tcmd_lat`, `trddata_en`, `tctrlupd_min`, `tctrlupd_max`, `tctrlupd_interval`, `tlp_resp`, `tlp_wakeup`	Deadlines or guarantees the MC provides to the PHY.
Both/system	`twrdata_delay`, `terror_resp`, `tinit_start`, `tinit_complete`, parity/CRC latencies	Coordinated timing across both sides.

19.2 Parameter families

Timing parameter families

Family	Purpose
Command timing	`tcmd_lat`, `tctrl_delay` relate the command bus to chip select and to the DRAM pins.
Write data timing	`tphy_wrlat`, `tphy_wrdata`, `twrdata_delay` schedule write data relative to commands.
Read data timing	`trddata_en`, `tphy_rdlat`, `tphy_rdcsgap`, `tphy_rdcslat` schedule read data return.
Update timing	`tctrlupd_*`, `tphyupd_type*`, `tphyupd_resp` govern update handshakes.
Status/init timing	`tinit_start`, `tinit_complete`, `tdram_clk_disable`, `tdram_clk_enable`, parity/alert latencies.
Training timing	`trdlvl_*`, `twrlvl_*`, `tcalvl_*`, `tphylvl*` pace training sequences.
Low power timing	`tlp_resp`, `tlp_wakeup` bound the low-power handshake.
Error timing	`terror_resp` bounds error reporting delay.

19.3 Minimum and maximum conventions

Most DFI timing parameters specify a minimum supportable value (often 0 or 1) and leave the maximum implementation-specific. The MC and PHY negotiate the actual values at integration time. The only hard rule is that the MC must meet the minimum assertion times and response deadlines, and the PHY must not exceed the maximum latencies it advertises.

19.4 Frequency-ratio timing

For frequency-ratio systems, timing parameters are defined in terms of DFI PHY clock cycles. In a matched-frequency system the DFI PHY clock and the DFI clock are the same, so the distinction disappears.

Source: DFI Specification v3.1 sections 3.1-3.8, 5.0

20 Ratio and Phasing

DFI 3.1 supports three frequency relationships between the MC clock and the PHY clock: 1:1, 1:2 and 1:4. The ratio affects command scheduling, signal naming and read data ordering.

20.1 Matched frequency (1:1)

In a 1:1 system the MC clock and the PHY clock are the same. There is one command opportunity per DFI clock cycle and one read/write data word per cycle. No `_pN` or `_wN` suffixes are needed, though phase 0 suffixes are optional.

20.2 1:2 frequency ratio

The PHY clock runs at twice the MC clock frequency. In each MC clock cycle the PHY can execute two commands or transfer two read data words. The MC therefore provides:

- `dfi_address_p0`, `dfi_address_p1`
- `dfi_cs_n_p0`, `dfi_cs_n_p1`
- `dfi_wrdata_en_p0`, `dfi_wrdata_en_p1`
- `dfi_rddata_en_p0`, `dfi_rddata_en_p1`
- `dfi_rddata_w0`, `dfi_rddata_w1` on the return path
- `dfi_rddata_valid_w0`, `dfi_rddata_valid_w1`

The PHY must be able to accept a command on any phase.

20.3 1:4 frequency ratio

The PHY clock runs at four times the MC clock frequency. The same replication pattern applies with `_p0` through `_p3` for commands and `_w0` through `_w3` for read data. The 1:4 mode is explicitly defined in DFI 3.1 even though the specification text often shows only `_p0/_p1` examples.

20.4 Read data rotation

In a frequency-ratio system, read data words return on `dfi_rddata_wN` in a rolling order. The MC and PHY must keep their data word pointers synchronized. A frequency change resets both pointers to zero.

20.5 Scheduling impact

Frequency ratio gives the MC two useful abilities:

1. Multiple command slots per MC clock. A 1:4 system can issue up to four commands in one MC clock cycle.
2. Higher data bandwidth at the PHY without increasing MC clock frequency.

The cost is wider buses, more complex pointer management and the need to drive enables in every phase for signals such as `dfi_cke` and `dfi_odt`.

20.6 Practical note for scoria

The scoria controller inherits a datapath designed around a four-phase unit of transfer (`DFI_RATE = 4`). The v2.1.1 specification enumerated the phase variants explicitly; v3.1 generalizes the notation to `_pN`. A naive signal-name diff would wrongly report the phase signals as removed. They are still present; only the naming convention changed.

Source: DFI Specification v3.1 sections 3.5.3, 4.8

21 Parameter Tables

This table lists every timing parameter named in the DFI 3.1 inventory, with a plain-English meaning and the interface it belongs to.

Complete DFI 3.1 timing parameter table

Parameter	Interface	Meaning
`tcmd_lat`	Control	Cycles from `dfi_cs_n` assertion to associated CA signals driven.
`tctrl_delay`	Control	Cycles from control signal change to that change reaching the DRAM interface.
`tphy_wrlat`	Write data	Cycles from write command to `dfi_wrdata_en` assertion.
`tphy_wrdata`	Write data	Cycles from `dfi_wrdata_en` assertion to `dfi_wrdata` driven.
`twrdata_delay`	Write data / Idle	Cycles from `dfi_wrdata_en` assertion to completion of write data transfer on DRAM bus.
`tphy_wrcslat`	Write data	Cycles from write command to `dfi_wrdata_cs_n` assertion.
`tphy_wrcsgap`	Write data	Additional cycles between commands when changing target write chip select.
`tphy_crcmin_lat`	Write data / CRC	Min cycles from CRC data on `dfi_wrdata_en` to CRC error on

Parameter	Interface	Meaning
		dfi_alert_n.
tphy_crcmax_lat	Write data / CRC	Max cycles from CRC data on dfi_wrdata_en to CRC error on dfi_alert_n.
trddata_en	Read data	Cycles from read command to dfi_rddata_en assertion.
tphy_rdlat	Read data	Max cycles from dfi_rddata_en assertion to dfi_rddata_valid assertion.
tphy_rdcslat	Read data	Cycles from read command to dfi_rddata_cs_n assertion.
tphy_rdcsgap	Read data	Additional cycles between commands when changing target read chip select.
tctrlupd_min	Update	Minimum cycles dfi_ctrlupd_req must be asserted.
tctrlupd_max	Update	Maximum cycles dfi_ctrlupd_req may be asserted.
tctrlupd_interval	Update	Maximum cycles between dfi_ctrlupd_req assertions.
tphyupd_type0	Update	Max cycles for PHY-initiated update type 0.
tphyupd_type1	Update	Max cycles for PHY-initiated update type 1.

Parameter	Interface	Meaning
tphyupd_type2	Update	Max cycles for PHY-initiated update type 2.
tphyupd_type3	Update	Max cycles for PHY-initiated update type 3.
tphyupd_resp	Update	Max cycles from dfi_phyupd_req to dfi_phyupd_ack.
tinit_start	Status / Init	Max cycles for PHY to acknowledge frequency-change request.
tinit_complete	Status / Init	Max cycles for PHY to re-assert dfi_init_complete after dfi_init_start de-asserts.
tdram_clk_disable	Status / Clock	Cycles from dfi_dram_clk_disable assertion to DRAM clocks held constant.
tdram_clk_enable	Status / Clock	Cycles from dfi_dram_clk_disable de-assertion to first valid DRAM clock edge.
tparin_lat	Status / Parity	Cycles from command to associated dfi_parity_in.
tphy_paritylat	Status / Parity	Max cycles from dfi_parity_in to dfi_alert_n assertion.
trdlvl_en	Training / Read	Min cycles from read training enable to first read/MRR command.
trdlvl_resp	Training / Read	Max cycles from dfi_rdlvl_req/dfi_rdlv

Parameter	Interface	Meaning
		l_gate_req to enable assertion.
trdlvl_max	Training / Read	Max cycles the MC waits for dfi_rdlvl_resp.
trdlvl_rr	Training / Read	Min cycles between read training reads/MRRs.
twrlvl_en	Training / Write	Min cycles from write leveling enable to first dfi_wrlvl_strobe.
twrlvl_resp	Training / Write	Max cycles from dfi_wrlvl_req to dfi_wrlvl_en.
twrlvl_max	Training / Write	Max cycles the MC waits for dfi_wrlvl_resp.
twrlvl_ww	Training / Write	Min cycles between dfi_wrlvl_strobe assertions.
tcalvl_en	Training / CA	Min cycles from dfi_calvl_en to dfi_cke de-assertion.
tcalvl_capture	Training / CA	Cycles from calibration command to dfi_calvl_capture pulse.
tcalvl_resp	Training / CA	Max cycles from dfi_calvl_req to dfi_calvl_en.
tcalvl_max	Training / CA	Max cycles the MC waits for dfi_calvl_resp.
tcalvl_cc	Training / CA	Min cycles between calibration commands.
tphylvl	Training / PHY request	Max cycles dfi_phylvl_req_cs_n

Parameter	Interface	Meaning
		stays asserted after acknowledge.
tphylvl_resp	Training / PHY request	Max cycles from PHY training request to MC acknowledge.
tlp_resp	Low power	Max cycles from low-power request to dfi_lp_ack.
tlp_wakeup	Low power	Max cycles from request de-assertion to dfi_lp_ack de-assertion.
terror_resp	Error	Max cycles from affected transaction to dfi_error assertion.

21.1 Programmable parameters

DFI 3.1 programmable parameters

Parameter	Meaning
`phycrc_mode`	`0` = MC generates CRC; `1` = PHY generates CRC.
`phydbi_mode`	`0` = MC handles DBI; `1` = PHY handles DBI.
`phyrdlvl_en`	PHY supports read data eye training.
`phyrdlvl_gate_en`	PHY supports gate training.
`phywrlvl_en`	PHY supports write leveling.
`phycalvl_en`	PHY supports CA training.

Source: DFI Specification v3.1 sections 3.1-3.8, Tables 5-22

22 Low Power Behavior

DFI 3.1's low-power interface is an opportunity request: the MC suggests that the PHY may save power, but the PHY decides whether to take the opportunity. The handshake does not directly enter DRAM self-refresh or power-down; those are DRAM commands issued through the control interface.

22.1 Low-power request split

v3.1 splits the v2.1 `dfi_lp_req` into:

- `dfi_lp_ctrl_req`: no more commands on the control interface.
- `dfi_lp_data_req`: no more activity on the data interface.

This allows the PHY to sleep the command path and data path separately. For example, a state that still refreshes memory can keep the control path awake while the data path sleeps.

22.2 Typical sequence

1. The MC issues the DRAM command to enter the desired low-power state (power-down enter or self-refresh enter) on the control interface.
2. The MC asserts `dfi_lp_ctrl_req` and/or `dfi_lp_data_req` with an appropriate `dfi_lp_wakeup` value.
3. The PHY may acknowledge with `dfi_lp_ack` within `tlp_resp` cycles.
4. The low-power state persists while request and acknowledge are both asserted.
5. When the MC needs to resume, it de-asserts the request.
6. The PHY de-asserts `dfi_lp_ack` within `tlp_wakeup` cycles and normal operation resumes.

22.3 Relationship to DRAM self-refresh

Self-refresh is a DRAM command, not a DFI state. The DFI low-power interface merely tells the PHY that the controller will not be sending commands or data for some time. The PHY may use that information to gate its own clocks or reduce power, but the DRAM refresh timing is handled by the DRAM's self-refresh logic once the self-refresh command is issued.

22.4 Scoria note

The scoria controller exposes `dfi_lp_ctrl_req` and `dfi_lp_data_req` for spec compliance, but the target PHY family does not consume them. CKE is driven as a CSR bit set during initialization, and the generated core does not implement a self-refresh or power-down engine.

Source: DFI Specification v3.1 sections 3.7, 4.12

23 Update Behavior

Updates are short pauses in normal traffic that let the PHY perform calibration or timing adjustments. DFI 3.1 supports both MC-initiated and PHY-initiated updates.

23.1 MC-initiated update

The MC asserts `dfi_ctrlupd_req` whenever it knows the DFI bus will be idle. The PHY may acknowledge with `dfi_ctrlupd_ack` or ignore the request. The request must last at least `tctrlupd_min` and no more than `tctrlupd_max` cycles, and the MC must issue a new request at least every `tctrlupd_interval` cycles.

While the request (and any acknowledge) is asserted, the DFI bus remains idle except for update-related activity.

23.2 PHY-initiated update

The PHY asserts `dfi_phyupd_req` with a valid `dfi_phyupd_type` when it needs idle time. The MC must respond with `dfi_phyupd_ack` within `tphyupd_resp` cycles. The request stays asserted until the update is done and must de-assert before the selected `tphyupd_typeX` expires.

The MC must not assert `dfi_ctrlupd_req` and `dfi_phyupd_ack` simultaneously. If both request signals are asserted at once, the PHY may withdraw its own request.

23.3 Idle-bus definition

The DFI bus is idle when:

- The control interface is not sending commands.
- All read data has been transferred to the MC.
- All write data has been transferred on the DFI bus and completed on the DRAM bus.
- No pages are open.

The `twrdata_delay` parameter defines how long after `dfi_wrdata_en` the write data is guaranteed to have completed on the DRAM bus. This prevents an update from cutting off a write burst.

23.4 Update after self-refresh exit

After exiting self-refresh, the MC must assert `dfi_ctrlupd_req` before resuming normal read or write traffic. This gives the PHY a chance to re-calibrate after the low-power state.

Source: DFI Specification v3.1 sections 3.4, 4.6

24 Frequency Change

DFI 3.1 reuses the initialization handshake for frequency changes during normal operation. The MC requests a change with `dfi_init_start`; the PHY acknowledges by de-asserting `dfi_init_complete`.

24.1 Acknowledged frequency change

1. The MC asserts `dfi_init_start` during normal operation.
2. The PHY de-asserts `dfi_init_complete` within `tinit_start` cycles to accept the request.
3. The MC holds `dfi_init_start` asserted while the clock frequency changes.
4. The MC de-asserts `dfi_init_start` when the new frequency is stable.
5. The PHY re-initializes for the new frequency and re-asserts `dfi_init_complete` within `tinit_complete` cycles.

Both sides reset their DFI read data word pointers to zero when `dfi_init_start` asserts.

24.2 Not-acknowledged frequency change

If the PHY does not de-assert `dfi_init_complete` within `tinit_start` cycles, the request is ignored. The MC must then de-assert `dfi_init_start` and may try again later.

24.3 Stable levels during the change

While `dfi_init_start` is asserted or `dfi_init_complete` is de-asserted, both the MC and the PHY must hold the DFI and memory interface signals at valid and stable levels. The DFI specification does not define a maximum overall duration for the frequency change.

24.4 Scoria note

The scoria controller implements the frequency-change protocol because it is part of the v3.1 initialization interface, but the DDR3/LPDDR3 target use case does not rely on dynamic frequency scaling.

Source: DFI Specification v3.1 sections 3.5.4, 4.9

25 Refresh Interaction

Refresh is a DRAM command issued on the DFI control interface. DFI 3.1 does not define refresh timing itself, but it does constrain when refresh can be scheduled relative to training, updates and low-power states.

25.1 Refresh and training

The MC must complete all in-flight transactions before starting training. Refresh therefore competes with training for idle-bus windows. DFI 3.1 adds support for refresh during training, meaning the MC is not required to disable refresh entirely while a training operation is in progress, but the details depend on the PHY and DRAM capabilities.

25.2 Refresh and update

An update requires the bus to be idle. A refresh command is a command, so it cannot overlap an update window. The MC must either:

- Finish the refresh before asserting `dfi_ctrlupd_req` or acknowledging `dfi_phyupd_req`, or
- Defer the refresh until the update handshake completes.

25.3 Refresh and low power

In self-refresh, the DRAM handles its own refresh. The DFI low-power interface only informs the PHY that the controller will be idle. When the MC exits self-refresh, it must issue `dfi_ctrlupd_req` before resuming normal traffic.

In power-down, refresh is suspended. The MC must ensure that the power- down duration does not violate the DRAM refresh requirements.

25.4 Scoria note

The scoria controller does not implement a self-refresh or power-down engine in the generated core. CKE is controlled by software through a CSR. Refresh commands are issued by the controller's normal scheduler on the DFI control interface.

Source: DFI Specification v3.1 sections 4.6, 4.11.6, 4.12

26 Revision Deltas

DFI 3.1 is best understood as the v3.0 DDR4 additions plus the LPDDR3 and interface-split changes of 3.1. The scoria controller inherits the v2.1.1 baseline from pumice and upgrades to 3.1 for the leveling and initialization model.

26.1 What carried over unchanged from v2.1.1

- Frequency ratios and the `_pN` replication convention. v2.1.1 enumerated phase variants explicitly; v3.1 generalizes them to `_pN`. The 1:4 ratio is still defined.
- The update interface (`dfi_ctrlupd_*` and `dfi_phyupd_*`).
- The basic command/address/data interface, apart from the DDR4 additions.
- `dfi_init_start` and `dfi_init_complete`, now reused for frequency change.
- `dfi_data_byte_disable` and `dfi_dram_clk_disable`.
- Read and write leveling as concepts, though the signal names and per-CS scheme were reworked.

26.2 v2.1.1 -> v3.0 changes

v3.0 added the features that enable DDR4:

v3.0 additions over v2.1.1

Area	v3.0 additions
Command encoding	`dfi_act_n`, `dfi_bg`, `dfi_cid`; `dfi_ras_n`/`dfi_cas_n`/`dfi_we_n` carry A16/A15/A14 when `dfi_act_n` is asserted.
Parity/CRC/alert	`dfi_parity_in`, `dfi_alert_n_aN`, CRC timing parameters.
DBI	`dfi_rddata_dbi_n`, `phydbi_mode`.
Data path chip select	`dfi_wrdata_cs_n`, `dfi_rddata_cs_n`.
Error interface	`dfi_error`, `dfi_error_info`, `terror_resp`.
Programmable parameters	`phycrc_mode`, `phydbi_mode`, per-training enables.

26.3 v3.0 -> v3.1 changes

v3.1 focused on LPDDR3 and interface refinement:

v3.1 additions over v3.0

Area	v3.1 additions
LPDDR3 CA training	`dfi_calvl_*`, `dfi_phy_calvl_cs_n`, CA training parameters.
LPDDR3 command support	LPDDR3 added to gate training, read data eye training and write leveling.
Low power split	`dfi_lp_req` becomes `dfi_lp_ctrl_req` and `dfi_lp_data_req`.
PHY-requested training	`dfi_phylvl_req_cs_n`, `dfi_phylvl_ack_cs_n` for non-DFI training mode.
LPDDR3 CA mapping	20-bit `dfi_address` maps to LPDDR3 CA bus.

26.4 Brief v4.0 preview

DFI 4.0 later extended the spec with a much larger training interface, including DBI training, PHY-master training, and additional status and frequency-change capabilities. Those features are beyond the scope of a DDR3/LPDDR3 controller and are not used by scoria.

26.5 What scoria leaves out

The scoria DDR3/LPDDR3 controller does not implement:

- DDR4 command encoding (`dfi_act_n`, `dfi_bg`, `dfi_cid`).
- CA parity, CRC and `dfi_alert_n`.
- DBI.
- LPDDR3 CA training (`dfi_calvl_*`).

These are either DDR4-era or LPDDR4-era features that do not apply to the target memory types.

Source: DFI Specification v3.1 Release Information and revision notes; `projects/components/mem-ctrl-ip/research-ip/scoria-ddr3-lpddr3/docs/scoria_has/ch04_interfaces/01_dfi_v31.md`

27 DFI 3.1 in the Scoria Controller

This chapter connects the DFI 3.1 specification to the scoria DDR3/LPDDR3 memory controller as documented in `projects/components/mem-ctrl-ip/research-ip/scoria-ddr3-lpddr3/docs/scoria_has/ch04_interfaces/01_dfi_v31.md`.

27.1 Why scoria chose v3.1

Scoria targets DDR3 and LPDDR3. Its predecessor pumice used DFI v2.1.1. The upgrade to v3.1 was driven by DDR3 write leveling: v3.1's per-chip- select leveling scheme is what future DDR4 work will also need, so paying for it once in scoria is cheaper than re-implementing it later.

27.2 What scoria inherits

The datapath is inherited almost unchanged from pumice:

- One async-FIFO clock-domain crossing, bubble-free.
- The unit of transfer is a DFI word.
- `DFI_RATE = 4`.

A naive signal-name diff between v2.1.1 and v3.1 might claim that phase variants such as `dfi_address_p2` were removed. They were not. v2.1.1 listed them explicitly; v3.1 generalized the notation to `_pN`. The 1:4 frequency ratio remains a defined mode.

27.3 What scoria implements from v3.1

Scoria DFI 3.1 implementation highlights

Feature	Scoria implementation
Write leveling	`scoria_wrlvl_ifc` drives `dfi_wrlvl_*`; firmware-driven search, not a hard-coded state machine.
Data path chip select	`dfi_wrdata_cs_n` and `dfi_rddata_cs_n` are driven; at `NUM_RANKS = 1` the value is constant rank 0.
`dfi_reset_n`	Driven from the init sequencer and presented as `dfi_reset_n_o` at the DFI boundary.
Low power requests	Exposed for spec compliance, but not consumed by the target PHY.
Error interface	`dfi_error` is used as a generic flag; DDR4-specific `dfi_error_info` encodings are ignored.

27.4 What scoria does not implement

DFI 3.1 features deliberately unimplemented in scoria

Group	Signals	Reason
DDR4 command encoding	`dfi_act_n*`, `dfi_bg*`, `dfi_cid*`	DDR3 has no dedicated ACT pin, bank groups or chip ID.
CA parity, CRC, alert	`dfi_alert_n*`, `dfi_parity_in_p*`	DDR4-era features.
Data bus inversion	`dfi_rddata_dbi*`	DDR4/LPDDR4 feature.
CA training	`dfi_calvl_*`, `dfi_ca_capture`, `dfi_phy_calvl_cs_n`	LPDDR3 CA training is treated as LPDDR4-era for this controller.

27.5 Known documentation corrections

The scoria tie-in document notes a few spec-related traps:

- The v2.1.1 -> v3.1 signal diff must not be trusted blindly; phase signals are renamed, not removed.
- `dfi_reset_n` is gated on memory type, not DFI version, so any scoria DFI claim implies this signal.
- `dfi_wrdata_cs_n` / `dfi_rddata_cs_n` are genuinely v3.1-only and are driven even in single-rank mode.

27.6 Study corrections carried into this book

The caveats in the DFI 3.1 inventory are worth remembering:

- Section 3.6 refers to a `dfi_ca_capture` pulse; the actual signal is `dfi_calvl_capture`.
- Section 4.11 contains typo variants such as `dfi_phlvl_ack_cs_n`; the defined names are `dfi_phylvl_req_cs_n` and `dfi_phylvl_ack_cs_n`.
- `dfi_rdlvl_edge` and timer-based signals such as `dfi_rdlvl_delay_x` appear only in revision-history text, not in the v3.1 signal tables.

Source: `projects/components/mem-ctrl-ip/research-ip/scoria-ddr3-lpddr3/docs/scoria_has/ch04_interfaces/01_dfi_v31.md`; DFI Specification v3.1 sections 3.6, 4.11